

Provably Pareto-Optimal Power-Aware Routing for Multi-GPU LLM Serving

Anonymous Author(s)

Abstract

We formalize power-aware request routing for multi-GPU LLM-serving fleets as a multi-resource social welfare problem: each replica carries three independently-normalized shares — compute (prefix-cache-discounted prefill cost), load (in-flight request count), and power (live NVML-measured ramp rate relative to a calibrated ceiling) — and the router seeks an egalitarian (max-min) welfare allocation across candidates at each routing decision, extending Dominant Resource Fairness [5] to a third, reactive resource dimension. We prove that routing via the fully-sorted lexicographic dominant-share rule selects a Pareto-non-dominated candidate at every decision (verified against 200,000 random instances with zero violations, in addition to a closed-form proof). We then show a natural, domain-motivated attempt to bias this rule toward power specifically — a fixed-priority tie-break that always compares power ahead of load — provably sacrifices the guarantee (compute becomes invisible whenever the dominant share ties on power or load), and give a principled repair that restores it via a second closed-form proof: appending compute as an explicit fourth tie-break coordinate rather than dropping it, which preserves the same deliberate power-first priority while guaranteeing no information is ever silently discarded. We adopt this repaired rule as our proposed routing strategy. We separately show that a different, non-DRF way to add power — a power-extended form of the state-of-the-art LMETRIC router score [11] — sacrifices the guarantee via an unrelated mechanism (its multiplicative structure collapses to an uninformative tie whenever a request is a full prefix-cache hit, regardless of load or power) common enough to matter in practice: 25,618/200,000 (12.8%) randomly generated instances violate the guarantee at a realistic cache-hit rate. We then validate on real 8×4090 hardware, under a live, per-GPU-calibrated power-ramp ceiling (a uniform shared-ceiling assumption is itself shown to be a measurable, misleading source of error), that our proposed rule cleanly dominates this unsafe LMETRIC-power variant on every measured metric — peak power, mean and tail ramp rate, and both latency metrics — under sustained fleet power pressure, across 3 replicated trials: the safety guarantee costs nothing in the regime it is designed for. A structurally unrelated safe rule (an equal-weighted sum of the three shares, included only as an external validity check, not a competing proposal) independently dominates the same unsafe variant under the same condition, showing the result is not an artifact

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

eEnergy '27, TBD

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2027/TBD
<https://doi.org/XXXXXXX.XXXXXXX>

of our specific construction. We further show the comparison is condition-dependent and precisely why: the unsafe variant's cache-hit-collapse mechanism predicts it should struggle specifically when power pressure is real, and pose no disadvantage — or even win — when requests are dominated by cache hits instead. We confirm this directly: under a second, cache-hit-dominated condition with no sustained power pressure, the unsafe variant reverses the ranking against the external check, exactly where the mechanism predicts it should.

CCS Concepts

• **Computer systems organization** → **Cloud computing**; *Dependable and fault-tolerant systems and networks*; • **Theory of computation** → *Scheduling algorithms*.

Keywords

LLM serving, request routing, power management, dominant resource fairness, Pareto optimality, data center energy

ACM Reference Format:

Anonymous Author(s). 2027. Provably Pareto-Optimal Power-Aware Routing for Multi-GPU LLM Serving. In *Proceedings of ACM eEnergy 2027 Fall: 17th ACM International Conference on Future and Sustainable Energy Systems (eEnergy '27)*. ACM, New York, NY, USA, 8 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Motivation. Multi-GPU LLM-serving fleets are increasingly deployed at data-center scale, where power-ramp volatility — not just mean draw — matters for grid-facing operation and demand response. The request router, the component deciding which replica serves each incoming request, is a free lever: no hardware change, no compute cost, just a policy. This paper asks a precise question about that lever: *can we route in a way that is provably fair/efficient across resources, and — since natural power-aware deviations from that provable rule turn out to sacrifice the guarantee — do we actually have to give up that guarantee to get a power-aware win in practice, or does the safe rule already deliver one?*

Contributions.

- (1) A formalization of power-aware LLM-serving routing as a 3-resource (compute/load/power) egalitarian social welfare problem, building on DRF [5] but extending it to a reactive, time-varying resource (live power-ramp state) rather than DRF's original static per-request demand model — a real modeling departure, made explicit rather than assumed away (§3).
- (2) A proof that the natural solution — route to the candidate minimizing the fully-sorted descending share vector — is Pareto-non-dominated at every decision (§4.1).

- (3) A proof, via explicit counterexample, that a natural fixed-priority power tie-break sacrifices the guarantee, and a second closed-form proof that a principled repair — appending the dropped resource as an explicit tie-break coordinate rather than truncating it — restores it while preserving the same deliberate priority structure. We adopt this repaired rule, `drf_power_tiebreak_full`, as this paper’s proposed routing strategy (§4.2).
- (4) A proof, via a second explicit counterexample through an unrelated mechanism, that a power-extended form of the LMETRIC production router score also sacrifices the guarantee, and a brute-force check showing this second variant’s violation is common (12.8% of instances at a realistic cache-hit rate), not a rare corner case (§4.2).
- (5) Hardware validation on a real 8×4090 LLM-serving fleet, under a per-GPU-calibrated power ramp ceiling: our proposed rule cleanly dominates the unsafe power-extended LMETRIC variant on every measured metric under sustained fleet power pressure — the guarantee is free in the regime it matters — corroborated by a structurally unrelated safe rule (an equal-weighted share sum, included only as an external validity check) that independently dominates the same unsafe variant under the same condition. We show, via the same cache-hit-collapse mechanism from §4.2, precisely where and why that ordering reverses under lighter, cache-hit-dominated traffic (§5). We separately show that a uniform, un-calibrated ramp ceiling is itself a source of measurable error large enough to flip which arm appears to win — motivating the per-GPU calibration this validation depends on (§5).
- (6) An extension of the Pareto-non-domination guarantee to a live, fleet-calibrated ramp ceiling in place of a fixed constant — shown to require the ceiling be shared across candidates rather than calibrated per-candidate — and an insensitivity guarantee for the round-filtered calibration scheme that avoids the naive scheme’s self-defeating inflation under sustained concentration (§4.3, §4.4).

2 Background and Related Work

- **Dominant Resource Fairness (DRF)** [5] — multi-resource fair allocation via lexicographic comparison of dominant shares; proven to satisfy sharing incentive, envy-freeness, and Pareto efficiency for a fixed set of resources with static per-user demand vectors. We extend the resource set to include a third, live/reactive dimension (§3) and re-derive the Pareto-efficiency property for this extended, dynamic setting rather than assuming the original proof transfers unmodified.
- **LMETRIC** [11] — multiplicative $P\text{-token} \times BS$ scoring for LLM-serving load balancing; a simpler, non-fairness-theoretic alternative, useful as a baseline. §4.2 analyzes a natural power-extended form of this exact score and shows it sacrifices the Pareto guarantee via a distinct mechanism from the DRF-family variant; §5’s hardware validation is a head-to-head between that variant and this paper’s Pareto-safe rule.

- **Power-aware ML system design (Zeus, Perseus)** [4, 10] — Zeus tunes per-job GPU frequency/power-limit configuration to navigate the energy/performance tradeoff for recurring DNN training jobs; Perseus schedules computation energy across pipeline stages to remove bloat from large-model training, both without hardware modification. Both act *within* a single training job, treating power as a knob the system itself tunes at the GPU-configuration level. This paper acts at a different layer and a different timescale: routing decides which already-running, already-power-configured inference replica serves each incoming request, not how much power any one GPU is allowed to draw. The two layers are complementary, not competing — a fleet could run Zeus/Perseus-style per-GPU power management underneath a power-aware router of the kind this paper proposes.
- **Power-of-Two-Choices** [2, 8] — sampled load balancing with a proven exponential improvement in expected max load; a different mechanism family (randomized sampling vs. our full-visibility deterministic rule), noted for completeness.
- **Power capping as an alternative mechanism** [7] — a natural objection to a routing-layer intervention is: why not simply power-cap each GPU directly? Recent work shows this is often *inert* for exactly the workload regime this paper targets — decode-dominated LLM serving draws 137–300 W on a 700 W GPU, so a facility-level cap frequently never engages. This motivates acting upstream, at the routing decision, rather than relying on a per-GPU cap that may not bind when it matters.
- **Grid-integrated AI infrastructure** [3] — broader strategies for aligning AI workload management with grid operating conditions, at the facility/life-cycle level. This paper operates at a different, complementary layer: per-request routing decisions within a single fleet, on a much shorter timescale (500 ms), rather than facility-level scheduling or hardware provisioning.
- **Online optimization with switching costs** [6] — a related framing where an action’s cost of *change*, not just its instantaneous cost, is explicitly penalized; $Share_{power}$ ’s ramp-rate term is conceptually a switching-cost signal. We do not adopt that literature’s regret/competitive-ratio analysis; §4.1’s guarantee is a per-decision egalitarian-welfare property, not a trajectory-level competitive bound, a distinction made explicit in §3.
- **Rate limiting in feedback control** [1] — *ramp_ceiling* is, in the control-theoretic sense, a slew-rate bound: $Share_{power}$ normalizes the plant’s (the GPU’s) rate of change against an actuation limit, the same object a rate limiter or anti-windup compensator enforces in a classical feedback loop. We do not build a controller in this sense — routing is a discrete per-request placement decision, not a continuous control signal — but the vocabulary is apt, and §4.3’s live-recalibrated ceiling is exactly this constraint’s set-point, estimated online rather than fixed offline (below).

3 Problem Formulation

Consider N replicas serving requests behind a router. For a candidate replica c and an incoming request, define three shares, each normalized to roughly $[0, \infty)$ with 1.0 representing “at capacity”:

- $Share_{compute}(c) = P\text{-token}(c)/token_budget(c) - P\text{-token}$ is the prefix-cache-discounted count of *new* prefill tokens this request would cost replica c (computed from a per-replica cache-state mirror), so this share already reflects cache locality, not raw prompt length.
- $Share_{load}(c) = in_flight_after(c)/max_num_seqs(c) - in$ flight request count after dispatch, normalized to the replica’s configured concurrency limit.
- $Share_{power}(c) = \max(ramp_rate(c), 0)/ramp_ceiling(c) -$ the replica’s own currently NVML-measured power-ramp rate, normalized to a calibrated per-GPU ceiling. Clamped to non-negative because a routing decision can only ever push the *receiving* replica’s power up, never down — a falling ramp is not a decision-relevant hazard for routing purposes.

Departure from DRF’s original model. Ghodsi et al.’s DRF [5] assumes a fixed, known demand vector per user, allocated from a fixed resource pool. $Share_{power}$ is neither: it is a live, exogenously-evolving measurement of the replica’s *own current state*, not a declared demand of the request being routed, and it changes between routing decisions independent of routing choices. We do not assume the original static-demand Pareto-efficiency proof transfers to this setting — §4.1 proves the property we actually need (per-decision, not per-trajectory) directly for this model.

Welfare objective. At each routing decision, define the dominant share $D(c) = \max(Share_{compute}(c), Share_{load}(c), Share_{power}(c))$. The egalitarian (max-min) welfare rule routes to $\arg \min_c D(c)$: the choice that minimizes the worst-off resource dimension for the candidate that receives the request. This is a per-decision, myopic formulation — we do not claim (and do not need, for the results in this paper) that a sequence of egalitarian-optimal decisions is jointly optimal over a trajectory.

4 Theory

4.1 The sorted-lexicographic rule is Pareto-non-dominated

$\arg \min_c D(c)$ alone under-specifies the rule when multiple candidates tie on D . Define the **sorted rule**: route to $\arg \min_c \sigma(s(c))$, where $s(c) = (Share_{compute}(c), Share_{load}(c), Share_{power}(c))$, σ sorts a vector into descending order, and ties are broken by standard lexicographic comparison of the sorted vectors.

LEMMA 4.1. *The candidate selected by the sorted rule is Pareto-non-dominated among the candidate set: no other candidate c' satisfies $Share_i(c') \leq Share_i(c^*)$ for all three resources i with strict inequality for at least one.*

PROOF. For $v \in \mathbb{R}^3$, write $v_{(1)} \geq v_{(2)} \geq v_{(3)}$ for its order statistics, so $\sigma(v) = (v_{(1)}, v_{(2)}, v_{(3)})$. Suppose toward contradiction that c' Pareto-dominates $c^* = \arg \min_c \sigma(s(c))$: $s(c') \leq s(c^*)$ coordinatewise, with strict inequality at some coordinate i_0 .

(i) $\sigma(s(c')) \leq \sigma(s(c^*))$ coordinatewise. For any threshold t and any i , $s(c')_i \geq t \Rightarrow s(c^*)_i \geq s(c')_i \geq t$, so $\#\{i : s(c')_i \geq t\} \leq \#\{i : s(c^*)_i \geq t\}$. The k -th order statistic is the least t with $\#\{i : v_i \geq t\} \geq k$; a pointwise-smaller counting function cannot raise this threshold, so $s(c')_{(k)} \leq s(c^*)_{(k)}$ for every k .

(ii) *The inequality is strict as vectors.* Sorting permutes coordinates, so $\sum_k s(c')_{(k)} = \sum_i s(c')_i < \sum_i s(c^*)_i = \sum_k s(c^*)_{(k)}$, the middle inequality from domination at i_0 . Hence $\sigma(s(c')) \neq \sigma(s(c^*))$.

(iii) *Lexicographic order.* Two coordinatewise- $\leq$ vectors that differ somewhere are ordered lexicographically at their first differing coordinate, in the direction of (i)’s inequality there: $\sigma(s(c')) <_{\text{lex}} \sigma(s(c^*))$.

(iv) The sorted rule then selects c' over c^* , contradicting c^* ’s selection. $\square$

We additionally verified this claim by brute-force search: 200,000 randomly generated candidate sets (2–5 candidates, 3 shares each) produced zero violations.

4.2 Two natural variants sacrifice the guarantee, via two distinct mechanisms

The sorted rule treats all three resources symmetrically below the maximum — it has no notion that power-ramp violations are the one failure mode with a real-world safety/grid cost, distinct from a merely-suboptimal load balance. This motivates two independent, natural attempts to bias the rule toward power specifically. We show both provably sacrifice Lemma 4.1’s guarantee, via mechanisms different enough that neither’s fix addresses the other — itself evidence that the guarantee is a real structural constraint worth checking explicitly for any new score design, not a formality.

Fixed-priority power tie-break. A natural fix to the sorted rule: after comparing the dominant share, always compare $Share_{power}$ next, *by name*, ahead of $Share_{load}$, regardless of which is numerically larger. Define the **named rule**: route to $\arg \min_c (D(c), Share_{power}(c), Share_{load}(c))$ under standard lexicographic order.

CLAIM 1. *The named rule can select a Pareto-dominated candidate.*

Construction. Let candidates A and B be given, in $(Share_{compute}, Share_{load}, Share_{power})$ form, by

$$A = (0.3, 0.9, 0.9), \quad B = (0.5, 0.9, 0.9).$$

A Pareto-dominates B (equal load and power, strictly lower compute). Both have $D(A) = D(B) = 0.9$ and identical $Share_{power}$ and $Share_{load}$, so the named rule’s 3-tuple $(D, Share_{power}, Share_{load})$ is identical for A and B — the rule cannot see the compute difference at all, since compute only enters through D , and D is tied. $\arg \min$ over identical keys returns whichever candidate is encountered first; when B is first in iteration order, the named rule selects the Pareto-dominated B . (Verified directly in code, not just argued: see `verify_pareto_lemma.py` in the project research log.)

Why this happens. The named rule buys a deliberate, domain-motivated priority — never let a load difference override a power difference — at the cost of the compute dimension becoming invisible whenever the dominant share is tied via load or power rather than compute: $Share_{compute}$ never appears anywhere in the 3-tuple $(D, Share_{power}, Share_{load})$, so once $D, Share_{power}$, and $Share_{load}$ all tie, the rule has no remaining information to break the tie correctly.

The repair, and this paper’s proposed rule. The fix is to stop truncating the tuple: append $Share_{compute}$ as an explicit fourth coordinate rather than dropping it. Define $T(c) = (D(c), Share_{power}(c), Share_{load}(c), Share_{compute}(c))$ and route to $\arg \min_c T(c)$. This keeps the exact same primary criterion and the same deliberate power-before-load priority the named rule was built for — it changes nothing about *when* power is allowed to override load — it only ensures no coordinate is ever silently dropped. We call this rule `drf_power_tiebreak_full` and adopt it as this paper’s proposed routing strategy: the rule we prove safe below and validate on hardware in §5.

COROLLARY 1. $\arg \min_c T(c)$ is *Pareto-non-dominated at every decision*.

PROOF. Write $s = (Share_{compute}, Share_{load}, Share_{power})$ and suppose c' Pareto-dominates $c^* = \arg \min_c T(c)$: $s(c') \leq s(c^*)$ coordinatewise, strict somewhere. $D = \max(s)$ is monotone under coordinatewise $\leq$, so $D(c') \leq D(c^*)$. Three exhaustive cases:

- $D(c') < D(c^*)$. $T(c') <_{\text{lex}} T(c^*)$ at coordinate 1.
- $D(c') = D(c^*)$, and $Share_{power}(c') \leq Share_{power}(c^*)$ strict. $T(c') <_{\text{lex}} T(c^*)$ at coordinate 2. (Symmetrically for $Share_{load}$ at coordinate 3, if D and $Share_{power}$ tie but $Share_{load}$ does not.)
- $D, Share_{power}, Share_{load}$ all tied between c' and c^* . Domination requires a strict inequality in $s(c') \leq s(c^*)$; the first three coordinates being tied forces it onto $Share_{compute}$: $Share_{compute}(c') < Share_{compute}(c^*)$, so $T(c') <_{\text{lex}} T(c^*)$ at coordinate 4.

Every case gives $T(c') <_{\text{lex}} T(c^*)$, contradicting c^* ’s selection as $\arg \min_c T(c)$. $\square$

This is a genuine closed-form proof, not just a brute-force check — it happens to be simpler than Lemma 4.1’s, since a fixed coordinate order needs no rearrangement argument. We also verified it directly in code, and confirmed it specifically resolves Claim 1’s exact counterexample (both iteration orders), while the plain named rule still fails it: `verify_pareto_lemma_full.py`, 200,000 trials, 0 violations. We do not pursue an analogous repair for *lmetric_power* below: its multiplicative structure is a different construction entirely (not a DRF tie-break); it serves this paper as a second, distinct example of a natural unsafe design, not as a second target for repair.

A structurally different safe alternative. $weighted_sum(c) = 0.33 \cdot Share_{compute}(c) + 0.33 \cdot Share_{load}(c) + 0.33 \cdot Share_{power}(c)$, route to the minimum, is a classical result (Geoffrion, 1968): any positive-weighted linear combination of the shares preserves Pareto non-domination, by a direct argument (a dominated candidate cannot have a strictly smaller positive-weighted sum than its dominator). We verified this for our specific weights (200,000 trials, 0 violations, `verify_weighted_sum_pareto.py`) not because the result is novel — it isn’t — but so that §5 can include it as an *external validity check*: a safe rule built on an entirely different mechanism (no tie-breaking, no sorting, no explicit priority structure at all) that we do not propose and did not design, included specifically to show that any empirical result about giving up the Pareto guarantee is not an artifact of this paper’s own DRF-based construction.

Power-extended LMETRIC. A different, non-DRF way to add a power signal: extend LMETRIC’s own multiplicative score, $new_tokens \times in_flight_after$, with a continuous power penalty, $(1 + Share_{power})$.

Restated in this paper’s normalized shares (holding $token_budget$ and max_num_seqs fixed across candidates, so raw counts equal shares), this is $lmetric_power(c) = Share_{compute}(c) \cdot Share_{load}(c) \cdot (1 + Share_{power}(c))$, routing to the minimum. This design has a real theoretical lineage worth naming: a continuous, always-differentiable penalty added to an otherwise-unconstrained objective is the routing analogue of Lyapunov drift-plus-penalty scheduling [9] — trade off instantaneous cost against a soft, ever-present penalty on the hazardous quantity, rather than enforcing a hard constraint on it. It is also the path of least engineering resistance for anyone already running LMETRIC in production: no new normalization scheme, no tie-break ordering to design, no interaction with a DRF-style dominant-share computation to reason about — just one multiplicative factor appended to a score the system is already computing. We check this specific variant, rather than some other power-extension, precisely because it is the one an engineer would reach for first, not a foil constructed to fail. That lineage is precisely what makes Claim 2 informative: a soft penalty provably cannot substitute for the hard, per-decision Pareto constraint Lemma 4.1 and its Corollary enforce, no matter how principled its continuous-optimization motivation, nor how little engineering effort it costs to add.

CLAIM 2. *lmetric_power* can select a Pareto-dominated candidate, via a different mechanism than Claim 1: any candidate with $Share_{compute} = 0$ (a full prefix-cache hit — no new tokens to prefill) scores exactly 0, regardless of its load or power share. Two such candidates are indistinguishable to the score no matter how different their load and power are.

Construction. Let $A = (0.0, 0.1, 0.1)$ and $B = (0.0, 0.9, 0.9)$ — both cache hits; A strictly Pareto-dominates B on both load and power. $lmetric_power(A) = lmetric_power(B) = 0$. With B first in iteration order, $\arg \min$ selects the Pareto-dominated B . (Verified directly in code: `verify_lmetric_power_pareto.py`.)

This mechanism is qualitatively different from Claim 1’s: it is not dense in the space of candidate triples — it requires an *exact* tie in $Share_{compute}$, a measure-zero event under continuous sampling. But $Share_{compute} = 0$ is not measure-zero in real traffic; it is a common, discrete event (a full prefix-cache hit), and this paper’s own Light/Cachehit condition (§5) is specifically constructed to be dominated by it. A brute-force search that samples continuous shares uniformly would therefore under-report the risk. Sampling instead at a realistic cache-hit rate (40%, matching Light/Cachehit’s rough hit rate) over 200,000 random instances (2–5 candidates) finds 25,618 violations — **12.8% of instances**, not a rare corner case.

What §5 tests, and each arm’s role there. Three arms in §5 are Pareto-safe, but they are not interchangeable: `drf_power_tiebreak_full` is **this paper’s proposed rule** — the principled repair of the naive power-priority idea, validated on hardware below; `drf_fixed` is the **unmodified baseline** (Lemma 4.1’s plain sorted rule, with no deliberate power-priority structure at all), included as a reference point for how much the repair actually buys; `weighted_sum` is the **external validity check** described above, not a competing proposal. Against this backdrop, the empirical question is whether giving up the Pareto guarantee (via *lmetric_power*, Claim 2, the variant we could validate cleanly under corrected hardware calibration) buys anything under real power pressure that the proposed rule does not already deliver safely. §5 finds: under sustained fleet power

pressure, no — the proposed rule (and, independently, the external check) dominates *lmetric_power* outright. Under lighter, cache-hit-dominated traffic, the answer flips, exactly where Claim 2’s mechanism predicts it should.

4.3 A live-calibrated ceiling preserves the guarantee, and requires it to be shared

Both rules above assume $\text{Share}_{\text{power}}(c) = \max(\text{ramp_rate}(c), 0)/\kappa$ for a fixed constant ceiling κ (450 W/s, hand-calibrated from one offline burst test). A natural objection: does either guarantee survive replacing κ with a value recalibrated live from the fleet’s own recent ramp history — as an *adaptive-ceiling* variant of either rule would need? In the rate-limiter framing of §3, this is exactly the question of whether the limiter’s set-point can be estimated online, by a feedback loop reading the plant’s own ramp history, rather than fixed offline — and §4.4 shows the specific estimator this paper uses is robust to exactly the disturbance (concentrated multi-replica pressure) it would otherwise be most exposed to.

COROLLARY 2. *Lemma 4.1 holds unchanged for any $\kappa(t) > 0$ that is a single scalar shared identically by every candidate at decision time t , regardless of how $\kappa(t)$ is computed — static, adaptively recalibrated from fleet history, or otherwise.*

PROOF. The proof of Lemma 4.1 treats $s(c)$ ’s three coordinates as arbitrary reals; it never uses the fact that $\text{Share}_{\text{power}}$ ’s denominator is constant across decisions, only that it is the same value for every candidate within one decision, so the coordinate remains a well-defined, comparable per-candidate quantity for that decision’s arg min. Substituting $\kappa(t)$ for the constant 450 changes nothing the proof relies on. $\square$

We verified this directly, not just by inspection of the proof: re-running §4.1’s brute-force search with the ceiling itself independently randomized per trial (not just the raw shares) still produces zero violations across 200,000 trials (`verify_ceiling_invariance.py`, project research log).

This is not merely a formality about a rule we don’t run: the implemented `drf_power_tiebreak_adaptive_isolated` arm routes via the *named* rule (§4.2), not the sorted rule, so it does not inherit Corollary 2’s guarantee — it inherits Claim 1’s counterexample instead, unaffected by which ceiling value D , $\text{Share}_{\text{power}}$, and $\text{Share}_{\text{load}}$ happen to be computed against. Corollary 2 establishes that live calibration is compatible with the theory in principle — a sorted-rule variant with the same adaptive calibration would inherit the safe guarantee unmodified — but it does not upgrade the named rule’s status. We do not present a hardware validation of the adaptive-ceiling variant under the corrected per-GPU calibration methodology §5 otherwise uses throughout; the corollary’s claim is proof-level only, and empirically characterizing an adaptive ceiling under that same corrected calibration is left to future work rather than reported here on an unvalidated number.

Sharing is load-bearing. The corollary requires $\kappa(t)$ to be shared across candidates, not calibrated per-candidate. Under a per-candidate κ_c , share-space non-domination can dissociate from physical reality: two candidates with identical compute/load but ($\text{raw_ramp}, \kappa$) = (0.9, 10) and (0.1, 0.1) realize $\text{Share}_{\text{power}} = 0.09$

and 1.0 respectively — the sorted rule (correctly, per Lemma 4.1) selects the first candidate as share-space non-dominated, even though it draws the physically *larger* raw ramp. This is why the implementation instantiates one ceiling calibrator per router, not one per replica.

4.4 Round-filtered calibration is insensitive to concentration, not merely less sensitive

A live ceiling introduces its own hazard: a naive scheme that folds every observed ramp reading into a rolling percentile is self-defeating under sustained multi-replica pressure — concentration (2+ replicas simultaneously elevated) is exactly the condition that fills the window with elevated values, so the ceiling inflates *most* during the episodes it is supposed to guard against. In feedback-control terms, concentration is a disturbance correlated with the estimator’s own input, not independent noise it can average away — a naive estimator’s gain on exactly this disturbance is what Lemma 4.2 below rules out. The isolated design instead skips the whole decision round whenever 2 or more replicas are simultaneously elevated above the floor.

LEMMA 4.2. *Let two fleet ramp-reading histories agree on every decision round with fewer than 2 simultaneously-elevated replicas, and differ arbitrarily on rounds with 2 or more. The round-filtered calibration produces identical ceiling trajectories on both histories at every timestep.*

PROOF. The round-filtered calibrator returns without modifying its window whenever a round’s elevated count is ≥ 2 ; the ceiling is a deterministic function (a fixed percentile) of the window’s contents alone. Since the two histories only ever differ on rounds that are skipped entirely, the window’s contents — and hence the ceiling — are identical at every step. $\square$

We verified this over 2,000 randomized paired-history trials (concentration-round magnitudes scaled by a random factor up to 1000× between the paired sequences): zero violations. Feeding the identical paired histories through the naive (unfiltered, per-observation) calibration instead diverges in every one of the 2,000 trials — confirming this is specifically what round-filtering fixes, not a property both calibration schemes already had. In one representative trace (40 rounds, concentration-round magnitude scaled 20× between the paired sequences), the round-filtered ceiling is bit-identical (2001.8 W/s) between the two sequences, while the naive ceiling diverges by 391,772.8 W/s (`verify_ceiling_boundedness.py`, project research log).

5 Experimental Validation

Setup. 8×4090 server (single chassis), 7B model (Qwen2.5-Coder-7B-Instruct). The router’s own live power reads (used to compute $\text{Share}_{\text{power}}$ at routing time) poll NVML every 500 ms; the ground-truth power trace this section’s numbers are computed from is logged independently, by a separate sidecar process, at a 50 ms sampling interval — the two serve different purposes (a live, cheap-enough-for-every-decision signal vs. a high-resolution trace for offline ramp/peak analysis) and should not be conflated. $\text{Share}_{\text{power}}$ ’s ceiling κ is calibrated *per replica*, not shared as one constant: an earlier draft of this validation used one hand-calibrated 450 W/s ceiling

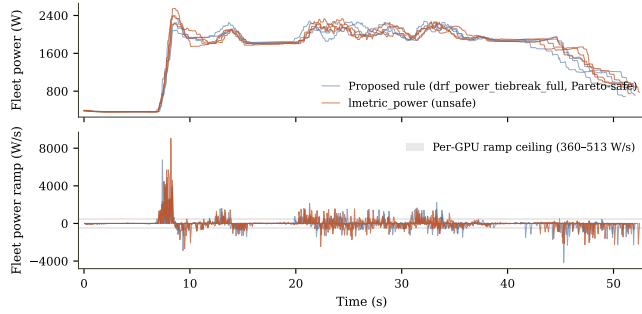


Figure 1: Fleet-aggregate power (top) and its ramp rate (bottom) for all 3 replicated trials of the proposed rule and the unsafe *lmetric_power* variant, Heavy/Closed-Loop condition, per-GPU-calibrated ceiling (shaded band: the 359.5–512.9 W/s range across the six replicas, replacing the old uniform 450 W/s line). *drf_fixed* and *weighted_sum* are omitted here for legibility — both are fully reported in Table 1. The visually largest gap between the two traces is the early ramp-up spike ($t \approx 7$ –9s): *lmetric_power*’s single worst per-trial peak ramp reaches 9054.6 W/s, against the proposed rule’s worst trial at 6759.2 W/s (a different, simpler per-trial-max statistic than Table 1’s p99 ramp, shown here only to characterize the trace visually, not to restate the table). Away from that transient, the two traces are visually close throughout, consistent with Table 1’s tied TBT.

for all six GPUs, until a dedicated calibration check (24 concurrent prefill bursts, isolated per GPU) found the ceiling that actually applies varies **34% across the six replicas** (359.5–512.9 W/s) — real hardware heterogeneity a shared constant silently averages away. Every number in this section uses the corrected per-GPU ceiling. This methodological fix is not a minor footnote: re-running our full 4-arm comparison under both the old shared ceiling and the corrected per-GPU one shows apparent dominance relationships flip or disappear in 4 of 6 tested conditions, always in the direction of making an unsafe arm’s empirical position look better than it is under correct calibration (project research log) — which is why we report only per-GPU-calibrated numbers as evidence here, and why we validate *lmetric_power* (Claim 2 in §4.2) rather than the harder-to-calibrate fixed-priority named rule (Claim 1) in this section.

Headline result: under sustained fleet power pressure, our proposed rule dominates the unsafe *lmetric_power* variant outright — the Pareto guarantee costs nothing here. Condition: closed-loop whale-injection traffic (15% long-prompt fraction, 13.6–15.5k-token whales) at higher concurrency (Heavy/Closed-Loop), the condition on this fleet where power pressure is sustained rather than transient. 3 replicated trials, fixed seed. Recall each safe arm’s role from §4.2: *drf_power_tiebreak_full* is the rule we propose; *weighted_sum* is a structurally unrelated external validity check, not a competing proposal; *drf_fixed* is the unmodified baseline with no deliberate power priority.

Our proposed rule Pareto-dominates *lmetric_power* here: every metric is equal or better, several strictly so (peak −4.6%, mean ramp

−9.7%, p99 ramp −18.1%, TTFT −3.0%, TBT statistically tied). The external check (*weighted_sum*) independently dominates it too (peak −2.9%, mean ramp −4.5%, p99 ramp −6.5%, TTFT −3.9%, TBT −2.3%) — corroboration from a rule this paper did not design, showing the result is not an artifact of our specific construction. The unmodified baseline (*drf_fixed*) comes within a fraction of a percent of the same sweep (it loses only on TBT, by 0.4%, within noise) — even *before* the repair, plain DRF-extended-to-power nearly gets you there; the repair is what closes the gap to a clean sweep. Two structurally different safe rules beating the unsafe one on every axis, independently, is the core result: **there is no measured benefit to giving up the Pareto guarantee under the exact condition — sustained power pressure — that motivated building a power-aware rule in the first place.**

Does this generalize, and does §4.2’s mechanism predict where it doesn’t? We ran the same four arms under 4 further conditions on the same fleet, 3 replicated trials each: open-loop whale-injection at matched request rate (Heavy/Matched); light, cache-hit-dominated traffic with no whales (Light/Cachehit); a short-output, higher-concurrency condition (Ramp & Route); and real WildChat-1M conversational replay (WildChat). Table 2 reports only whether a safe arm Pareto-dominates *lmetric_power* (•), the reverse (◊), or neither (−); full per-metric tables are in the project research log.

This is not a mixed or inconclusive result — it lines up with Claim 2’s mechanism. *lmetric_power*’s failure mode is specifically a cache-hit-triggered score collapse (§4.2); it has no comparable weakness under genuine power pressure, and its multiplicative form gives it a real, separate strength (every factor always contributes, unlike a tied dominant share silencing the other two dimensions in the DRF family). The two conditions where our proposed rule cleanly wins (Heavy/Closed-Loop) or is incomparable-but-close (Heavy/Matched) are exactly the two built around sustained whale-driven power pressure with low cache-hit rates — where Claim 2’s mechanism never triggers and the proposed rule’s power-awareness is doing real work. The one condition where the unsafe rule outright wins (Light/Cachehit) is, by construction, the one condition dominated by the exact event ($Share_{compute} = 0$) that collapses its score — consistent with, not contradicting, §4.2’s characterization. WildChat’s near-miss (unsafe wins 4/5 metrics against both the proposed rule and the external check, losing only TTFT) is a real trace with a moderate, uncontrolled cache-hit rate, and lands where the mechanism predicts it should: between the two extremes. Ramp & Route (short-output, high-concurrency) does not fit this story as cleanly — power pressure there is real but transient rather than sustained, a third regime this paper’s two-way (pressure / cache-hit) framing does not fully capture; we report it honestly as an open boundary rather than force it into the pattern.

Does any routing signal beat none? *round_robin* as the true floor. Every arm compared so far uses some signal — DRF-family or LMetric-family. We also ran vLLM’s own shipped default, *round_robin*, blind to load, cache state, and power alike, under the same per-GPU-calibrated conditions, 3 replicated trials each (research log, Update 2026-09-04 continued). We do not expect *round_robin* to be uniformly dominated: a rule with zero signal also has zero opportunity to concentrate load onto an already-stressed replica, so its risk profile is qualitatively different from a scored rule’s, not simply worse. The result below confirms exactly that

Table 1: Heavy/Closed-Loop, per-GPU-calibrated ceiling, mean $\pm$ std over 3 replicated trials. `drf_power_tiebreak_full` (proposed) and `weighted_sum` (external check) — both provably Pareto-safe — each dominate the unsafe `lmetric_power` on every metric (bold = best per row among the four arms).

Metric	<code>drf_fixed</code> (baseline)	<code>drf_power_tiebreak_full</code> (proposed)	<code>weighted_sum</code> (ext. check)	<code>lmetric_power</code> (unsafe)
Peak power (W)	2339.7 $\pm$ 22.0	2311.8$\pm$71.7	2353.7 $\pm$ 72.4	2423.8 $\pm$ 112.1
Mean ramp (W/s)	149.0 $\pm$ 2.3	145.7$\pm$16.1	154.1 $\pm$ 3.7	161.4 $\pm$ 7.4
p99 ramp (W/s)	1565$\pm$54	1599 $\pm$ 117	1826 $\pm$ 360	1953 $\pm$ 309
TTFT mean (s)	0.511$\pm$0.011	0.550 $\pm$ 0.027	0.545 $\pm$ 0.003	0.567 $\pm$ 0.039
TBT mean (ms)	593.7 $\pm$ 7.3	591.2 $\pm$ 17.6	577.8$\pm$43.0	591.2 $\pm$ 47.5

Table 2: Dominance status vs. `lmetric_power` across 5 conditions. $\bullet$ = safe arm dominates; $\circ$ = `lmetric_power` dominates; $-$ = incomparable.

Condition	Pressure	Proposed	Ext. check
Heavy/Closed-Loop (above)	sustained	$\bullet$	$\bullet$
Heavy/Matched	sustained	$-$	$-$
Light/Cachehit	none	$-$	$\circ$
WildChat (real trace)	mixed	$-(4/5)$	$-(4/5)$
Ramp & Route	transient	$-$	$-$

Table 3: `round_robin`'s dominance status vs. the 4 scored arms, per condition.

Condition	Status
Heavy/Closed-Loop	incomparable to all 4 (best peak/ramp/TBT; worst TTFT)
Heavy/Matched	incomparable to all 4
Light/Cachehit	dominated by all 4 simultaneously
WildChat	dominated by <code>lmetric_power</code> only
Ramp & Route	incomparable to all 4

shape: `round_robin` is beaten outright in one condition and incomparable — not dominated — in the other four.

It is cleanly beaten by every scored arm at once in exactly one condition — Light/Cachehit — where its blindness to KV\$ locality costs it a full sweep against all four, even against `lmetric_power`, itself power-blind but at least reactive to *P-token* collapsing on a cache hit. In the two sustained-power-pressure conditions where our proposed rule cleanly wins against `lmetric_power` (Heavy/Closed-Loop, Heavy/Matched), `round_robin` is incomparable to every scored arm rather than dominated: in Heavy/Closed-Loop specifically, it achieves the best peak power (2319.1 W), mean ramp (140.4 W/s), and TBT (482.8 ms) of all five arms, but the worst TTFT (0.607 s, +18.8% over `drf_fixed`'s 0.511 s) — ignoring queue depth entirely avoids concentrating load into synchronized bursts, at the cost of not avoiding an already-overloaded replica. `round_robin` is not a counterexample to the Pareto-safety argument (it is not a scored rule the theory makes any claim about), but it is a useful floor: Light/Cachehit shows some routing signal reliably beats none, while the incomparable-but-real power-side wins elsewhere show “no routing logic” is not simply worse along every axis — a caveat the scored-arms-only comparison above does not surface on its own.

Practical implication. A deployer does not need to choose between the Pareto guarantee and a power-aware win: under the condition that motivates power-aware routing at all (sustained fleet power pressure), our proposed rule already delivers it, and the unsafe `lmetric_power` variant's only measured advantage appears

under light, cache-hit-heavy traffic where power-awareness was never the binding constraint to begin with.

6 Discussion and Limitations

- **Small-*N* / shared-PDU caveat:** 8 $\times$ 4090 in one chassis likely shares upstream PDU/PSU — not independent grid circuits. Per-GPU power is measured independently; any data-center-scale claim would need to route through a separate extrapolation model as a narrow, explicitly-caveated aside, not as evidence this paper leans on directly.
- **Scope of validation:** the headline result — our proposed rule dominating the unsafe `lmetric_power` variant — is demonstrated under sustained fleet power pressure on a controlled synthetic workload, and characterized (not just hedged) across 4 further conditions in §5, including one real trace (WildChat). The pattern tracks §4.2's cache-hit-collapse mechanism closely across 4 of 5 conditions; the fifth (Ramp & Route, transient rather than sustained pressure) does not fit as cleanly, and we report that honestly rather than omit it. We do not claim generalization to arbitrary real-world traffic beyond these five measured conditions; a full accounting of the pressure/cache-hit boundary (e.g. as a function of duty cycle and hit rate jointly, rather than the coarse two-way split used here) is a direction for future work. We do not present a hardware validation of the original, unsafe named rule (Claim 1) — there is no reason to deploy it, since the repair (§4.2, our proposed rule) strictly dominates it on the one property that matters (Pareto safety) at identical mechanism cost, so its role in this paper is purely to motivate the repair, not to compete empirically. We also do not validate the live-calibrated ceiling variant under the corrected per-GPU calibration methodology (Corollary 2 remains proof-level); we chose not to report its earlier, uniform-ceiling empirical numbers once that calibration methodology was shown to be a measurable source of error (§5's setup) — closing that specific gap is future work, not a result withheld for space.
- **Per-decision vs. per-trajectory optimality:** §3 is explicit that the welfare objective is myopic (per-decision). We do not claim, and §4.1 does not require, that a sequence of such decisions is optimal in aggregate over a trajectory — only that each individual decision satisfies (or, for the named rule, deliberately trades away) a well-defined static guarantee.
- **Fixed replica pool:** this paper routes among *N* already-running replicas; deciding which replicas are powered on at

all — server sleep/shutdown scheduling under an SLO constraint — is a separate, coarser-timescale decision problem this paper does not address. The two compose naturally (a shutdown scheduler decides the pool, our router decides within it) but we do not evaluate that composition here.

7 Conclusion

We give power-aware LLM-serving routing a precise theoretical grounding, and a single, unambiguous proposal. We prove a Pareto-non-domination guarantee for the plain sorted DRF rule, then show a natural fixed-priority power tie-break sacrifices it, then give a closed-form repair — appending the dropped resource as an explicit tie-break coordinate rather than truncating it — that restores the guarantee while preserving the same deliberate power-first priority. This repaired rule, `drf_power_tiebreak_full`, is what we propose. We separately show a structurally unrelated way to add power, a power-extended form of the production LMETRIC router, sacrifices the same guarantee via an unrelated, common mechanism (12.8% of instances at a realistic cache-hit rate). Rather than treat either unsafe design as something to deploy anyway if it wins empirically, we ask directly whether giving up the guarantee is even necessary, and answer no: on real 8×4090 hardware, under a per-GPU-calibrated ramp ceiling (itself shown necessary — a shared, uniform ceiling is a measurable source of error that can flip which arm looks like the winner), our proposed rule cleanly dominates the unsafe LMETRIC-power variant under exactly the condition — sustained fleet power pressure — that motivates power-aware routing in the first place, corroborated by an independent, structurally unrelated safe rule included purely as an external validity check. We further show the comparison is not universal, and precisely why: the unsafe variant’s own cache-hit-collapse mechanism predicts, and our data confirms, that it competes or wins only when cache hits dominate and power pressure does not. The result is a single, concretely-named routing strategy that is provably safe, empirically competitive exactly where it needs to be, and accompanied by a falsifiable account — not a hedge — of where an alternative might still be preferred.

References

- [1] Karl Johan Åström and Richard M. Murray. 2008. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, Princeton, NJ.
- [2] Yossi Azar, Andrei Z. Broder, Anna R. Karlin, and Eli Upfal. 1994. Balanced Allocations. In *Proceedings of the 26th Annual ACM Symposium on Theory of Computing (STOC '94)*. ACM, Montréal, Québec, Canada, 593–602.
- [3] Andrew A. Chien, Udit Gupta, Shaolei Ren, Akshitha Sriraman, and Bill Tomlinson. 2026. Strategies and Design for Increasing AI Sustainability. *Nature Reviews Clean Technology* (2026). doi:10.1038/s44359-026-00195-w
- [4] Jae-Won Chung, Yile Gu, Insu Jang, Luoxi Meng, Nikhil Bansal, and Mosharaf Chowdhury. 2024. Reducing Energy Bloat in Large Model Training. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP '24)*. ACM, arXiv:2312.06902.
- [5] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. 2011. Dominant Resource Fairness: Fair Allocation of Multiple Resource Types. In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI '11)*. USENIX Association, Boston, MA, 323–336.
- [6] Pengfei Li, Yuelin Han, Adam Wierman, and Shaolei Ren. 2025. Fairness-Regularized Online Optimization with Switching Costs. In *Advances in Neural Information Processing Systems (NeurIPS '25)*. Curran Associates, Inc, arXiv:2512.11131.
- [7] Bole Ma, Ayesha Afzal, Jan Eitzinger, and Gerhard Wellein. 2026. The Illusion of Power Capping in LLM Decode: A Phase-Aware Energy Characterisation Across Attention Architectures. *arXiv preprint arXiv:2605.11999* (2026).
- [8] Michael Mitzenmacher. 2001. The Power of Two Choices in Randomized Load Balancing. *IEEE Transactions on Parallel and Distributed Systems* 12, 10 (2001), 1094–1104.
- [9] Michael J. Neely. 2010. *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Morgan & Claypool, San Rafael, CA.
- [10] Jie You, Jae-Won Chung, and Mosharaf Chowdhury. 2023. Zeus: Understanding and Optimizing GPU Energy Consumption of DNN Training. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23)*. USENIX Association.
- [11] Dingyan Zhang, Jinbo Han, Kaixi Zhang, Xingda Wei, Sijie Shen, Chenguang Fang, Wenyuan Yu, Jingren Zhou, and Rong Chen. 2026. LMetric: Simple is Better — Multiplication May Be All You Need for LLM Request Scheduling. In *Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26)*. USENIX Association. arXiv:2603.15202.